

The Government-Contracting Operating System: An Open-Source Build Plan for Stormwind Contracting

TL;DR

- **Build the spine, bolt on the hunting.** Make a living business profile + adaptive roadmap the center of the system using a Git-versioned **markdown task store** (YAML/TOML frontmatter for status/dependencies, body for context) read/written by Claude Code and Codex, optionally backed by **Letta** (formerly MemGPT) for durable self-editing memory. Your existing SAM SQLite mirror + Elasticsearch RAG + FastMCP server become *tools* the spine calls, not the core.
- **Use Goose as the unattended orchestrator** — it has native MCP, cron-style `schedule add`, YAML "recipes," and can drive **Claude Code and Codex as ACP providers** — so you wire your existing stdio FastMCP server straight in and let scheduled jobs run the SAM pipeline overnight. Put a thin **claudecodeui/CloudCLI** web UI in front for the chat "button" passthrough.
- **A "harness" is a fixed scorer + a human-edited markdown "program" + one comparable metric + keep/discard.** For a no-ground-truth domain, manufacture ground truth from *your own past go/no-go decisions*: hand-label past SAM opportunities fit/monitor/reject, then use **promptfoo** (label dataset → accuracy/F1) or **DSPy GEPA** (score + textual feedback → evolves your criteria markdown) to keep or discard each proposed edit to your selection-criteria files.

Key Findings

1. **Orchestration:** Goose is the best fit for a time-constrained solo founder who already lives in MCP + CLI agents. LangGraph is the most production-grade but has a real learning curve; CrewAI is fastest to prototype; Microsoft AutoGen is effectively in maintenance mode and should be avoided for new builds. All support MCP now, but Goose uniquely combines scheduling, recipes, and ACP passthrough to Claude Code/Codex.
2. **Memory/profile:** Letta is the strongest "structured evolving profile + self-editing memory" engine and is self-hostable with an MCP server; Mem0 and Zep are lighter "memory-as-a-service" layers. But the simplest durable store is **plain markdown files in Git** — and that doubles as your roadmap.
3. **Roadmap/checklist:** No need to build this. Adopt a markdown-task-as-code convention (taskmd, MDTM, markdown-plan) where each task is a file with frontmatter (`status`, `priority`, `dependencies`, `tags`) and a body. Agents read/update them; Git is the state machine; "route around blockers" is just "work any task whose dependencies are met and which isn't `blocked`."

4. **GovCon data:** The official, free, authoritative sources are SAM.gov (Opportunities + Entity APIs via `api.data.gov`), USAspending.gov (award history / incumbent analysis, no key required), eCFR (FAR/CFR text via REST API), and SBA size standards (13 CFR 121.201). Existing open-source projects worth integrating/forking: **makegov/awesome-procurement-data**, **OpenSAM** (akshayakula/OpenSAM), and the makegov Tango SDKs.
5. **Email:** Use an existing IMAP MCP server (e.g., `ai-zero/lab/mcp-email-server` or `non-dirty/imap-mcp`) with an app-password, run it read-only, and trigger downstream agent actions via a scheduled Goose recipe that polls for SAM alert emails.
6. **Harness:** The autoresearch pattern transfers cleanly if you replace `val_bpb` with **agreement against your hand-labeled gold set** (macro-F1 + Cohen's kappa). `promptfoo` gives you a labeled-dataset accuracy harness in YAML; DSPy GEPA gives you an automated criteria-evolution loop driven by score + natural-language feedback.
7. **Architecture:** Spine (markdown roadmap + Letta) → Orchestrator (Goose recipes + schedules) → Tools (your FastMCP server, SAM SQLite, Elasticsearch RAG, email MCP, USAspending) → Agents (Claude Code/Codex via ACP) → UI (`claudecodeui` passthrough) → Harness (`promptfoo`/GEPA on labeled opportunities). OpenClaw stays as the multi-agent routing layer or is superseded by Goose where it overlaps.

Details

1. Agent orchestration / scaffolding (open source)

The honest landscape as of mid-2026:

Goose is the recommended backbone. Per the Linux Foundation's Dec. 9, 2025 announcement, Block donated Goose as a founding contribution to the new **Agentic AI Foundation (AAIF)** alongside Anthropic's MCP and OpenAI's AGENTS.md; the repo moved from `block/goose` to `github.com/aaif-goose/goose` (announced Apr. 7, 2026). It is Apache-2.0, written in Rust, ships a desktop app + CLI + embeddable API, and was the *first public MCP client*. The features that matter for you:

- **Native MCP** with one of the deepest integrations in the ecosystem (70+ documented extensions). Extensions (MCP servers) are added to `~/.config/goose/config.yaml` — your existing stdio FastMCP server drops in with a `command`/`args` block exactly like the GitHub example.
- **Scheduled/unattended jobs:** `goose schedule add --schedule-id daily-report --cron "0 9 * * *" --recipe-source ./recipes/daily-report.yaml`. The scheduler spins up a fresh agent per run.
- **Recipes:** YAML files bundling instructions, required extensions, parameters, retry logic, and sub-recipes — a clean way to encode "the nightly SAM hunt."
- **ACP provider passthrough:** Goose "works as an ACP server... and can use ACP agents like Claude Code and Codex as providers." This is the cleanest path to "route straight through to

Claude Code and Codex."

- **Lead-Worker model:** assign different models to planning vs. execution vs. review.

Tradeoffs vs. alternatives, decision-relevant:

- **LangGraph** — graph/DAG orchestration, built-in SQLite/Postgres checkpointing with "time-travel," deepest MCP integration (tools become graph nodes), best observability (LangSmith). It is the right call if you later need auditable, deterministic, branch-heavy workflows — but the graph/state learning curve is real and is probably overkill for a solo founder's v1.
- **CrewAI** — role-based ("crew") metaphor, fastest to a working prototype, added event-driven "Flows" in 2025. Good for quick experiments; less durable state than LangGraph.
- **Microsoft AutoGen** — now consolidating into AG2; multiple sources describe Microsoft's AutoGen as effectively in maintenance mode with focus shifted to a broader Agent Framework. Avoid for a new build; the AG2-vs-AutoGen-0.4 split is a needless tax.
- **Letta** — really a stateful-agent runtime (see §2), not a general orchestrator.
- **CLI agents themselves** (Claude Code, Codex CLI, OpenHands, Aider, OpenCode, Pi): These are the *workers* Goose orchestrates. **Codex CLI** is open-source (Apache-2.0, Rust), supports MCP over STDIO and HTTP (config in `~/.codex/config.toml`), has a headless `codex exec` mode for CI/scripting, `--full-access`/`--full-auto` for unattended runs, and reads `AGENTS.md`. **Claude Code** has headless/SDK modes, MCP, Skills, hooks, scheduled cron jobs (toggle `CLAUDE_CODE_DISABLE_CRON`), and an "Auto memory" feature that persists notes under `~/.claude`. **OpenHands** (All-Hands-AI, MIT-licensed) is the strongest self-hosted "autonomous, sandboxed, headless-in-CI" option if you want Docker-isolated unattended runs and a model-agnostic backend (LiteLLM, 100+ providers); at version 1.7.0 (May 2026) it had crossed 74,400+ GitHub stars, 9,400+ forks and 102 releases (maintainer raised an \$18.8M Series A), achieving 72.8% on SWE-Bench Verified with Claude Sonnet 4.5 on its V1 SDK harness.

MCP-over-stdio fit: Goose, Claude Code, and Codex all consume stdio MCP servers directly via a `command`+`args`+`env` config block. Your Dockerized FastMCP server exposing `search_opportunities`, `ingest_public_document`, `search_documents`, etc. plugs into all three with no code change.

2. Persistent memory / profile engine (open source)

Two viable layers, and a recommendation to combine them:

- **Letta (formerly MemGPT)** — the most capable "evolving profile" engine. Core concepts: a stateful agent = system prompt + **memory blocks** + messages + tools. Memory blocks are editable strings, can be **shared across agents**, and are self-edited by the agent via memory tools. Letta is self-hostable (Docker server on `localhost:8283`); there's a community

Letta-MCP-server (oculairmedia) exposing `get_core_memory`, `update_core_memory`, `create_block`, `attach_block`, etc. over MCP so Claude Code/Codex can read and write the profile. Newer **Letta Code** stores memory as a **git-backed filesystem (MemFS)** of markdown files under `~/.letta/agents/<id>/memory`, with `system/` files always loaded — i.e., it *natively* implements the "markdown profile in Git" pattern, plus `/sleep` "dreaming" for background consolidation and `/doctor` to audit memory drift.

Benchmarks (LongMemEval ~ low-to-mid; episodic coherence strong) suggest Letta excels at *long-horizon task coherence* (its strength is "yesterday we tried X and it failed"), which is exactly what an evolving business roadmap needs.

- **Mem0** — lightweight "smart cache between your app and the LLM"; large ecosystem (~53k stars), MCP server, generous free tier, self-hostable via OpenMemory (needs Docker + an embeddings key). Best if you want a drop-in fact store rather than a full agent runtime.
- **Zep / Graphiti** — temporal knowledge graph (tracks how facts change over time); Graphiti engine is open source but requires Neo4j, and the full Zep platform has moved to a credit-based SaaS model. More than you need.

Recommendation: Keep the **structured state (the checklist with statuses, the business profile) as markdown files in a Git repo** — it's transparent, diffable, grep-able, agent-readable with zero tooling, and survives any tool change. Layer **Letta** on top only if/when you want true self-editing long-term memory and background consolidation; its MemFS model means you don't even have to migrate formats.

3. Roadmap / checklist / task-state engine (the spine)

This is the heart of the request, and the strong finding is: **do not build a bespoke engine — adopt the markdown-task-as-code pattern**, which has independently converged across multiple projects:

- **taskmd** (driangle/taskmd): one markdown file per task in `./tasks`, YAML frontmatter (`id`, `status`, `priority`, `effort`, `type`, `dependencies`, `tags`), body for objective/subtasks/acceptance criteria. Has a Claude Code plugin and a `taskmd validate` CLI. Explicitly designed "for developers working with AI coding agents," local-first, no server.
- **MDTM** (Markdown-Driven Task Management, from roo-commander): TOML frontmatter (`+++`) with status/priority/assignee/parents/dependencies, Markdown body with checklists; "Git-Native... AI agents can directly read task files." Status doubles as an **ownership/workflow gate** (e.g., `draft` vs `approved`) — useful for "agent may only act on approved items."
- **markdown-plan** (rexgarland): plans parse into a **tree or DAG**; explicit dependencies via `@(unique-substring)` syntax; ships a parsing library and `mdplan plot` for DAG/burn-up visualization; cyclic dependencies are rejected as invalid.

- **Tasks.md** (BaldissaraMatheus): a self-hosted, Docker-deployable markdown-file kanban board if you want a visual layer over the same files.

How the required behaviors map:

- *Living profile + adaptive roadmap*: a `PROFILE.md` (or Letta memory block) + a `tasks/` directory. Your pending items (VetCert/SDVOSB, FinCEN BOI, eVA, SWaM, first micro-purchase bid) each become a task file with `status: pending|in-progress|blocked|done` and `dependencies`.
- *Natural-language add/remove/reprioritize*: "I don't want the small-business cert" → the agent deletes/sets `status: dropped` on `tasks/0xx-small-business-cert.md` and commits. Because it's just files + Git, every change is auditable and revertible.
- *Never hard-block on one item*: this is a **scheduling rule in your Goose recipe / agent instructions**, not a feature you build: "Each run, select all tasks whose `status` is `pending` and whose `dependencies` are all `done`; never spend more than one cycle on a `blocked` task; always advance at least N parallel workstreams." A DAG (markdown-plan) makes "which tasks are unblocked right now" a parsing operation.

The pending roadmap items, with authoritative specifics to seed the profile:

- **SDVOSB certification** is now done through **SBA's VetCert portal at veterans.certify.sba.gov** (the function moved from VA's CVE to SBA on Jan 1, 2023). It is **free**, requires 51%+ unconditional ownership/control by a service-disabled veteran (any rating, even 0%), small under the primary NAICS code, active SAM.gov registration, and documents like DD-214 + VA disability letter. Self-certification ended Dec 22, 2024 — certification is mandatory to count toward goals. Per SBA's Nov. 11, 2025 announcement, SBA "cleared the backlog of all pending applications, and reduced application processing times to an average of just 12 days" — down from 81 days at the end of 2024 (processing "averaged 30 days when the program began...skyrocketed to 81 days by the end of 2024"), cutting the backlog "from more than 2,700 to zero actionable cases." The SDVOSB federal goal rose from 3% to 5% under **P.L. 118-31 (FY2024 NDAA), enacted Dec. 22, 2023, Section 863**, which per Congressional Research Service report IN12313 "simply deletes the term '3 percent' in the underlying goaling statute, 15 U.S.C. 644(g)(1), and replaces it with '5 percent.'" SBA reports SDVOSBs received **\$31.9 billion (5% of federal contracting dollars) in FY2024**. The governing rule is **13 CFR Part 128**.
- **Virginia SWaM + state SDVOSB designation** runs separately through the **Virginia Department of Small Business and Supplier Diversity (DSBSD)**; the service-disabled designation requires a Virginia Department of Veterans Services (DVS) service-disabled veteran certification letter, and there is **no charge**.
- **Primary NAICS 541512** (Computer Systems Design Services) — verify the current size standard against SBA's table (13 CFR 121.201) when seeding the profile.

Free, official, authoritative APIs (prioritize these):

- **SAM.gov Get Opportunities API** (<https://api.sam.gov/prod/opportunities/v2/search>), docs at open.gsa.gov) — published opportunity details, requires an api.data.gov key, paginated, **active notices updated daily**. Per the GSA Get Opportunities API docs: "Rate limit for Federal User is 1000 requests/day; Rate limit for Non-Federal User is 10 requests/day." Non-federal users associated with a registered entity get 1,000/day; federal system accounts default to 10,000/day. This is a hard constraint that justifies your local SQLite mirror. [Gsa](#) [Gsa](#)
- **SAM.gov Entity API** — entity registration data (UEI, status, set-aside/cert flags) for incumbent/teammate research.
- **USAspending.gov API** (<https://api.usaspending.gov>) — **no API key required**; award history, recipient analytics, sub-awards, NAICS/PSC/agency breakdowns. This is your **incumbent analysis** engine ("who won this before, for how much"). Caveat: GAO has repeatedly flagged data-quality/completeness issues, so treat results as directional. The advanced-search endpoints ([spending_by_award](#)), ([spending_by_award_count](#)) take rich filter objects.
- **eCFR REST API** (<https://www.ecfr.gov/developers/documentation/api/v1>) — FAR/CFR text, updated daily, point-in-time versions; good for grounding FAR/13 CFR/48 CFR clauses in your RAG. (FAR is Title 48; SBA size standards are Title 13 Part 121.)
- **SBA size standards** — 13 CFR 121.201 (and the downloadable table at sba.gov/document/support-table-size-standards) for the fit logic.

Existing open-source projects (so you don't reinvent):

- **makegov/awesome-procurement-data** — a curated index of GovCon APIs/utilities (SAM, FPDS, CALC labor rates, PSC/NAICS prediction engines, "Pulse of GovCon" consolidating SAM+Challenge.gov+Grants.gov, NASA's Sec. 889 compliance tool, a USAspending Slack bot). Start here as a map.
- **OpenSAM** (akshayakula/OpenSAM, MIT) — Next.js app doing semantic search over SAM.gov opportunities + multi-LLM RAG chat + "upload past performance docs and chat with your data." Closest thing to your existing pipeline; worth studying/forking its adapter/RAG patterns, though it overlaps with what you've already built.
- **makegov Tango SDKs** (Node/Python/Go/Rust) — official client libraries for a federal-contracts API (contracts, IDVs, entities, opportunities, grants, vehicles). Useful if you want typed clients rather than raw HTTP.
- Note the **GitHub [federal-contracts](#) topic** already lists MCP servers in this exact space (e.g., "MCP server for MoltAwards: agent-native federal + state contract opportunities," tagged with [openclaw](#), [gov-contracting](#), [claude-skill](#)). Survey these before building new glue — some may be directly composable with your OpenClaw layer.

Verdict: Your SAM-SQLite-mirror + Elasticsearch-RAG stack already replicates the valuable core of OpenSAM and the commercial tools. Bolt on **USAspending** (incumbent/award history — the biggest free capability you're missing) and **eCFR** (clause grounding) next; both are keyless or low-friction.

5. Email ingestion

The clean open-source pattern is an **IMAP MCP server + app password + scheduled poll**:

- **ai-zerolab/mcp-email-server** (`uvx mcp-email-server@latest stdio`), also a Docker image) — IMAP+SMTP over MCP, TOML config, multi-account, attachment download; **read-only mode when SMTP is omitted** (compose tools hidden) — ideal for ingest-only safety. Env-var config for host/port/SSL.
- **non-dirty/imap-mcp** — Python(`uv`), OAuth2 support, (`allowed_folders`) to restrict access (e.g., just `INBOX`), designed to "act as intelligent email assistants."
- **n24q02m/better-email-mcp** — 6 composite tools, multi-account, auto-discovery, stdio + HTTP modes, explicit setup docs for Claude Code/Codex.

Security guidance from the sources: never use your main password — generate an **app-specific password**; store it in an environment variable, not a committed file; restrict to the minimum folder set; prefer read-only. **Trigger pattern:** a Goose recipe on a cron schedule ("every 30 min, search INBOX for SAM alert / agency notices, parse new ones into the SQLite store, and create/append task files for anything matching the criteria profile").

6. The harness + human feedback loop (the core learning goal)

What a "harness" actually is. In Karpathy's (`autoresearch`), the system has exactly three parts, and the discipline comes from keeping them separate:

1. **The harness** — (`prepare.py`): fixed code that prepares data and **scores** the result. The agent *is forbidden to edit it*. If the agent could edit the scorer, "it would just make the test easier instead of making the model better." This is the integrity lock.
2. **The program** — (`program.md`): a plain-English instruction file, "the research agenda written by a human... the only thing the human actively maintains across sessions." This is the human's control surface — the **knobs**.
3. **The thing being optimized** — (`train.py`): the file the agent edits.

The loop: read state → make one change → run experiment under a **fixed budget** → measure **one comparable metric** (`val_bpb`, lower is better) → **keep if improved, else `git reset`** → repeat. Karpathy's framing: "one GPU, one file, one metric." The README is explicit that "it works on anything you can score."

The hard part for GovCon: there is no `val_bpb`. "Is this opportunity a fit?" is a judgment call with no fast objective ground truth. The solution is to **manufacture ground truth from your own past decisions** and then borrow the established techniques for evaluating subjective LLM systems:

- **Build a labeled gold/backtest set.** Hand-label a few hundred past SAM opportunities as **fit** / **monitor** / **reject** — these are your `val` set. This is the single most valuable thing you can do, and it's the foundation everything else hangs off.
- **Pick one comparable metric.** Because most opportunities are "reject" (class imbalance), the recommended primary metric is **macro-averaged F1** (`sklearn.metrics.f1_score(..., average='macro')`), which per scikit-learn "simply calculates the mean of the binary metrics, giving equal weight to each class... macro-averaging may be a means of highlighting [infrequent-class] performance," so the rare "fit" class isn't drowned out. Report **per-class precision/recall** alongside, and use **Cohen's kappa** as the headline "agreement-with-Jeremy" number. A useful nuance: scikit-learn notes `cohen_kappa_score` "is intended to compare labelings by different human annotators, not a classifier versus a ground truth" — which actually *fits* your case, since you're measuring agreement between two raters (you vs. the criteria-file scorer). Pair it with macro-F1 because kappa can behave counterintuitively under heavy class imbalance (a documented critique, e.g., the PLOS One paper doi:10.1371/journal.pone.0222916).
- **LLM-as-judge** for the fuzzy parts (e.g., "does this capability statement match the solicitation?"), with eyes open to documented **verbosity and position biases** in LLM judges (Zheng et al. 2023, MT-Bench, arXiv:2306.05685).
- **Human-in-the-loop preference collection:** as new opportunities come in and you make real go/no-go calls, append them to the gold set — the harness gets sharper over time *for free*.

The tunable knobs (your control surface). These are the GovCon analog of `program.md`:

- Your **markdown selection-criteria / capability profiles** (`TECHNICAL_SERVICES_PROFILE.md`, `ELASTIC_LEAD_PROFILE.md`, lead-selection logic).
- **Scoring rubrics** (what counts as fit) and **thresholds** (e.g., minimum match score; NAICS allow-list; max/min contract value; set-aside filters; place-of-performance = DMV).
- The metric and the keep/discard threshold itself.

Concrete recommended harness design (the controllable loop Jeremy asked for):

Option A — promptfoo (simplest, declarative, recommended to start). promptfoo is open source (Apache-2.0 core), runs 100% locally, and is purpose-built for "test-driven prompt engineering." (OpenAI announced its acquisition of Promptfoo on March 9, 2026, stating "Promptfoo will remain open source under the current license"; it was founded in 2024 by Ian Webster and Michael D'Angelo and is used by 25%+ of the Fortune 500 with 350k+ developers.)

1. Export your labeled past opportunities to **CSV/JSON** (promptfoo loads test cases from `tests.csv`); supported formats include YAML/JSON, JSONL, CSV, TS/JS, Google Sheets).
2. Put the **expected label per row** in the special `__expected` column (multiple assertions via `__expected1/2/3`); for a 3-class label, a bare value defaults to the `equals` assertion, or use `llm-rubric:` for fuzzier grading. (Note: as of promptfoo ≥ 0.120 , `llm-rubric` defined in `defaultTest` does not resolve `{{var}}` templates — define per-test if the rubric needs the input.)
3. List **both versions of your criteria markdown** under the top-level `prompts:` key (`file://prompts/criteria_v1.md`, `file://prompts/criteria_v2.md`) — promptfoo renders them as **side-by-side columns scored on the same labeled set**, giving a directly comparable accuracy.
4. promptfoo's accuracy = "the proportion of prompts that produce the expected or desired output." Compute multi-class **precision/recall/F1 via the** `derivedMetrics` config block (the docs ship exact formulas, and there's an `eval-f-score` example repo). Gate keep/discard with the `PROMPTFOO_PASS_RATE_THRESHOLD` env var (the CLI returns exit code 100 when the pass rate is below threshold) — that's your automated "keep or discard."
5. The loop: agent proposes an edit to `criteria_v2.md` → `promptfoo eval` against your gold labels → if macro-F1/accuracy improves, keep (commit); else discard (`git reset`). **You** remain in control by owning the labels and the rubric.

Option B — DSPy GEPA (automated criteria evolution, the closest analog to autoresearch). GEPA (Genetic-Pareto reflective prompt evolution; Agrawal et al. 2025, arXiv:2507.19457) is the more powerful "self-improvement" engine. Its key property: the **metric function returns both a score and natural-language feedback** — `dspy.Prediction(score=float, feedback=str)` — so per the DSPy docs, "such textual feedback provides GEPA more visibility into why the system got the score that it did, and then GEPA can introspect to identify how to improve the score... propose high performing prompts in very few rollouts." The exact metric signature is `metric(gold, pred, trace=None, pred_name=None, pred_trace=None) -> float | ScoreWithFeedback`.

- You supply your labeled examples as the `trainset` (list of `dspy.Example`); per the docs, "GEPA uses the trainset to perform reflective updates to the prompt, but uses the **valset** for tracking Pareto scores."
- Your metric returns `score = 1.0 if pred == your_label else 0.0` plus feedback explaining the disagreement ("Labeled REJECT but criteria scored FIT — NAICS didn't match registered codes").
- GEPA evolves the `signature.instructions` (your editable criteria text) and **keeps candidates based on validation score** via a Pareto frontier ("maintains a Pareto frontier: the set of candidates which achieve the highest score on at least one evaluation instance"); `best_candidate`/`val_aggregate_scores` expose the winner. Configure with `dspy.GEPA(metric=..., reflection_lm=dspy.LM(model="gpt-5", temperature=1.0,`

```
max_tokens=32000), auto="medium") then (gepa.compile(student,
trainset=examples)).
```

- This automates the edit-and-keep step that you'd do by hand in Option A — but you still own the labels and can inspect/approve every evolved criteria file before it goes live.

Recommendation: Start with **promptfoo** (you'll have a working, comparable, label-driven harness in an afternoon, and it doubles as regression protection when you tweak prompts). Graduate to **DSPy GEPA** once your gold set is large enough (a few hundred labeled examples) and you want the criteria files to *evolve themselves* under your supervision. Keep both honest with the autoresearch discipline: **the harness/labels are the locked scorer; the markdown criteria are the program you edit; macro-F1/kappa is the one comparable metric; keep-or-discard on agreement.**

7. Putting it together — reference architecture

A layered design that maximizes reuse of your existing stack and minimizes custom glue:

Layer 0 — State (the spine), in one Git repo:

- `PROFILE.md` (living business profile: certs held/pending, NAICS, capabilities, DMV focus).
- `tasks/` directory of markdown task files (taskmd/MDTM convention; frontmatter `status`/`priority`/`dependencies`/`tags`). Seed with VetCert/SDVOSB, FinCEN BOI, eVA, SWaM, first micro-purchase bid.
- `criteria/` directory: `TECHNICAL_SERVICES_PROFILE.md`, `ELASTIC_LEAD_PROFILE.md`, lead-selection logic — these are the **harness knobs**.
- (Optional) Letta with shared memory blocks / MemFS pointing at the same markdown, for self-editing long-term memory + background "dreaming."

Layer 1 — Orchestrator: Goose.

- Register your **existing stdio FastMCP server** as a Goose extension in `config.yaml`.
- **Recipes** encode workflows: `roadmap-review.yaml` (read tasks, advance unblocked workstreams, never hard-block), `sam-hunt.yaml` (query SAM SQLite mirror + Elasticsearch RAG via your MCP tools, score against `criteria/`, append fit/monitor opportunities as task files), `email-ingest.yaml`, `incumbent-research.yaml` (USAspending).
- **Schedules** (`goose schedule add --cron`) run the hunt + email poll unattended overnight.

Layer 2 — Workers: Claude Code & Codex via ACP. Goose uses them as ACP providers for the heavy agentic work; both also speak MCP directly if you prefer to bypass Goose for interactive sessions.

- Your FastMCP server (`search_opportunities`, `ingest_public_document`, `search_documents`, ...).
- SAM SQLite daily mirror (works around the 10 req/day API limit).
- Elasticsearch RAG over solicitation PDFs.
- **New, low-effort additions:** USAspending (keyless incumbent analysis), eCFR (FAR/CFR grounding), an IMAP MCP email server (read-only, app password).

Layer 4 — Chat passthrough UI. Put `claudecodeui` / `CloudCLI` (`siteboon/claudecodeui`, free OSS, self-hostable) in front: a responsive web/mobile chat that reads/writes the same `~/.claude` config, manages MCP servers, exposes a shell, and supports **Claude Code, Codex, Cursor, and Gemini CLI**. This is your "button" — chat routes straight through to the CLI agents. (Alternatives: `sugyan/claude-code-webui` for a thinner single-session wrapper; Open WebUI if you want a multi-provider chat that talks to APIs rather than the CLIs.) [GitHub](#)

Layer 5 — Harness (separate, locked). A `harness/` directory with your labeled opportunities (CSV/JSON), a promptfoo config (and later a DSPy GEPA script), and the keep/discard metric (macro-F1 + Cohen's kappa). Run it on demand or on a schedule; it proposes edits to `criteria/*.md` that are kept only if agreement with your labels improves.

Where OpenClaw fits: OpenClaw (your Node/TS multi-agent framework; note there's also a public `openclaw.ai` ecosystem with an `openclaw-mcp` bridge to Claude.ai and an `openclaw-molt-mcp` FastMCP server) can remain the **routing/channel layer** that delegates tasks across agents, or the chat brain that "delegates tasks to a bot which spins up Claude Code." Where Goose's recipes/schedules/ACP overlap with OpenClaw's orchestration, prefer the one with less maintenance burden — for unattended scheduled GovCon jobs, Goose's batteries-included scheduler is hard to beat; keep OpenClaw for the bespoke multi-agent routing it already does well. [GitHub](#)

The few places you'll write custom glue (flagged honestly):

1. The **scoring/labeling harness logic** specific to opportunities (the promptfoo `derivedMetrics` for fit/monitor/reject, or the GEPA metric+feedback function). This is small but bespoke.
2. The **"route around blockers" scheduling rule** in your Goose recipe / agent instructions (a few lines of policy, not a system).
3. The **mapping from email/SAM events → task files** (a recipe + a little parsing).

Everything else — orchestration, memory, task format, CLI agents, UI, data APIs, email — is off-the-shelf open source.

Recommendations (staged, for a time-constrained solo founder with a W2)

Stage 1 — Spine + reuse (a weekend). Create the Git repo with `PROFILE.md`, `tasks/` (taskmd convention), and `criteria/`. Write the roadmap-review and never-hard-block policy as a markdown instruction file. You already have Claude Code/Codex + FastMCP; just point them at this repo. *Benchmark to advance:* agents reliably read/update task status and reprioritize from natural-language instructions.

Stage 2 — Orchestrate + schedule (a few evenings). Install Goose, register your FastMCP server as an extension, write `sam-hunt.yaml` and `roadmap-review.yaml` recipes, schedule them with cron. Add the IMAP email MCP (read-only, app password) and an `email-ingest.yaml` recipe. *Benchmark:* an overnight run produces a triaged list of new fit/monitor opportunities as task files without you touching the terminal.

Stage 3 — UI passthrough (an evening). Stand up claudecodeui/CloudCLI on your machine for the chat "button." *Benchmark:* you can drive the whole system from a phone browser on your LAN.

Stage 4 — Harness v1 (promptfoo). Hand-label ~150–300 past opportunities fit/monitor/reject. Build the promptfoo eval (`__expected` labels, `derivedMetrics` for macro-F1, two criteria-file columns). *Benchmark:* you can run `promptfoo eval` and get a single comparable macro-F1 across two versions of your criteria markdown. *Threshold that changes the plan:* if macro-F1 on your own held-out labels is below ~0.6, your criteria files (not the tooling) need work — iterate the markdown before automating.

Stage 5 — Self-improvement (DSPy GEPA), only once the gold set is large and stable. Wrap the criteria selection as a DSPy module, write the score+feedback metric, run GEPA to evolve `criteria/*.md` under your review. *Benchmark:* GEPA proposes a criteria edit that beats your hand-tuned version on held-out labels — and you approve it before it ships. *Threshold:* if GEPA's gains don't transfer to *new months'* opportunities, you're overfitting your past taste; shrink the trainset or add more recent labels.

Add incrementally, not up front: USAspending incumbent analysis and eCFR clause grounding (Stage 2–3); Letta self-editing memory (only if plain markdown + Git proves insufficient for long-horizon coherence).

Caveats

- **Rate limits are real:** SAM's non-federal Opportunities API is capped at **10 requests/day** (1,000/day for non-federal users tied to a registered entity) — your local SQLite mirror is essential, not optional. A federal system account (needs a .gov email) is the only path to ~10,000/day.
- **Data quality:** GAO has repeatedly documented completeness/accuracy gaps in USAspending; treat incumbent analysis as directional, not authoritative.

- **LLM-as-judge bias:** documented verbosity and position biases (Zheng et al. 2023) mean any LLM-graded portion of your harness should be spot-checked against your own labels; the human gold set is the anchor.
- **Cohen's kappa caveat:** scikit-learn frames kappa as inter-annotator (not classifier-vs-truth) agreement and it can mislead under class imbalance — report macro-F1 + per-class precision/recall alongside it.
- **Ecosystem churn:** the CLI-agent/eval space is moving fast (Goose moved to the Linux Foundation/AAIF; promptfoo was acquired by OpenAI in March 2026 but its OSS core stays Apache-2.0; AutoGen→AG2 split; one source notes a Gemini CLI retirement). Favor permissively licensed, self-hostable tools (Goose Apache-2.0, promptfoo Apache-2.0, DSPy, Letta) and keep your state in plain Git so any single tool is swappable.
- **Name collision:** "OpenClaw" is also a public third-party ecosystem (openclaw.ai) with its own MCP bridges; don't confuse those repos with your proprietary OpenClaw framework when searching for integrations.
- **Security:** unattended agents with shell/file/email access can do damage fast; use Codex/OpenHands sandboxing, read-only email, least-privilege MCP scopes, and Goose's permission controls. Don't expose claudecodeui to the public internet without auth. ([GitHub](#))
- **Certification dynamics:** SDVOSB processing times and goal percentages cited here reflect late-2025/early-2026 SBA announcements and the FY2024 NDAA; re-verify VetCert timelines and the current 13 CFR 121.201 size standard for NAICS 541512 at the time you actually file, since these change.